# LeetCode Practice

## 6 Hard: Sliding Window

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

**6**
**HARD**

**Patterns (in order)**

1. Sliding Window (6)

**How to use**

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

# Contents

**6 questions | tap a title to jump to it**


**Hard questions (6)** Go >

**Sliding Window (6)** Go >

H 1. Substring with Concatenation of All Words LC 30

H 2. Minimum Window Substring LC 76

H 3. Contains Duplicate III LC 220

H 4. Minimum Window Subsequence LC 727

H 5. Subarrays with K Different Integers LC 992

H 6. Longest Duplicate Substring LC 1044

## Hard questions (6)

HARD | TOPIC 1 OF 1

# Sliding Window

**6 Hard questions start on the next page**

**Before this:** this is the first topic
**After this:** this is the last topic

---

IN THIS TOPIC

**1.** Substring with Concatenation of All Words LC 30
**2.** Minimum Window Substring LC 76
**3.** Contains Duplicate III LC 220
**4.** Minimum Window Subsequence LC 727
**5.** Subarrays with K Different Integers LC 992
**6.** Longest Duplicate Substring LC 1044

HARD | Sliding Window | LeetCode #30 | 1 of 6 | 1/6

# Substring with Concatenation of All Words

## QUESTION

You are given a string s and an array of strings words. All the strings of words are of the same length.

A concatenated string is a string that exactly contains all the strings of any permutation of words concatenated.

CONSTRAINTS

- For example, if words = ["ab","cd","ef"], then "abcdef", "abefcd", "cdabef", "cdefab", "efabcd", and "efcdab" are all concatenated strings. "acdbef" is not a concatenated string because it is not the concatenation of any permutation of words.

Return an array of the starting indices of all the concatenated substrings in s. You can return the answer in any order.

**Example 1**

| | |
|---|---|
| Input | `s = "barfoothefoobarman", words = ["foo","bar"]` |
| Output | `[0,9]` |
| Explanation | *The substring starting at 0 is "barfoo". It is the concatenation of ["bar","foo"] which is a permutation of words. The substring starting at 9 is "foobar". It is the concatenation of ["foo","bar"] which is a permutation of words.* |

**Example 2**

| | |
|---|---|
| Input | `s = "wordgoodgoodgoodbestword", words = ["word","good","best","word"]` |
| Output | `[]` |
| Explanation | *There is no concatenated substring.* |

**Example 3**

| | |
|---|---|
| Input | `s = "barfoofoobarthefoobarman", words = ["bar","foo","the"]` |
| Output | `[6,9,12]` |
| Explanation | *The substring starting at 6 is "foobarthe". It is the concatenation of ["foo","bar","the"]. The substring starting at 9 is "barthefoo". It is the concatenation of ["bar","the","foo"]. The substring starting at 12 is "thefoobar". It is the concatenation of ["the","foo","bar"].* |

- $1 <= s.length <= 10^4$
- 1 <= words.length <= 5000
- 1 <= words[i].length <= 30
- s and words[i] consist of lowercase English letters.

## INTERVIEW APPROACH · STAR-LC

## PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find all starting indices in s where a concatenation of ALL words in words (any order) begins.

Each word used exactly once. Right?"

"A few quick questions:

All words same length? Yes. Word can repeat in words list? Yes."

"Let me walk: s='barfoothefoobarman', words=['foo','bar'] -> [0,9] ok

## PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for ordering in orderings(words):
    joined = concatenate it
    record every index where joined occurs in s
```

| TIME | SPACE |
|---|---|
| O(m! * n * m * k) | O(m * k) |

WHY IT IS SLOW

m words have m! orderings and each is searched in s. Slide a window of m words over s in steps of k, keeping a counter of the words seen: O(n * k) in total (k = word length).

*"Should I go ahead and optimize?"*

## PHASE 3 OPTIMIZE

*"Fixed-size sliding window with word frequency map — O(n*k). Walk-through:"*

`window_size = len(words[0]) * len(words).`

`word_len = len(words[0]). required = Counter(words).`

For each offset in 0..word_len-1 (k offsets to cover all alignments):

Slide a window of size len(words) words across s[offset:]:

Use a deque or two-pointer: current word frequency map.

For each position: extract s[pos:pos+word_len].

If word not in required: reset window from after this position.

**If word in required:**

Add to window. If window[word] > required[word]: shrink from left until equal.

If len(window_words) == len(words): record left position as answer.

**WHY k offsets:** words may start at any alignment mod word_len.

**Time:** O(n*k) where k = word length. Space: O(n).

## PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import Counter, defaultdict
class _30_SubstringWithConcatenation:
    def findSubstring(self, s, words):
        if not s or not words: return []
        k = len(words[0])
        m = len(words)
        total = k * m
        word_freq = Counter(words)
        result = []
        for offset in range(k):
            window = defaultdict(int)
            count = 0
            left = offset
            for right in range(offset, len(s) - k + 1, k):
                word = s[right:right + k]
                if word in word_freq:
                    window[word] += 1
                    count += 1
                    while window[word] > word_freq[word]:
                        window[s[left:left+k]] -= 1
                        count -= 1
                        left += k
                    if count == m:
                        result.append(left)
                        window[s[left:left+k]] -= 1
                        count -= 1
                        left += k
                else:
                    window.clear()
                    count = 0
                    left = right + k
        return result
```

## PHASE 5 TEST & DEBUG

*"Let me trace findSubstring on s='barfoothefoobarman', words=['foo','bar']."*

word_len=3, num_words=2, total=6, n=18

Start i=0 (window 'barfoo'): 'bar'->{bar:1}, 'foo'->{foo:1} == need -> valid, add 0

Start i=3 (window 'foothe'): 'foo'->{foo:1}, 'the'->not in words -> invalid

Start i=6 (window 'thefoo'): 'the'->not in words -> invalid

Start i=9 (window 'foobar'): 'foo'->{foo:1}, 'bar'->{bar:1} == need -> add 9

Start i=12: 'bar'->{bar:1}, 'man'->not in words -> invalid

Returns [0,9] ok

**WHY:** slide by word_len, rebuild word count each window -> O(n*total/word_len).

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n)** | **O(m)** |
| O(n/k) words per offset × k offsets = O(n) | window |

**Follow-up:** if words have variable lengths, the sliding window trick doesn't apply — need suffix arrays.

## PHASE 6 COMPLEXITY & FOLLOW-UP

**Follow-up:** Minimum Window Substring (#76) — single word target, same window idea.

IN YOUR APP | Expose | Editorial | Community

HARD | Sliding Window | LeetCode #76 | 2 of 6 | 2/6

# Minimum Window Substring

## QUESTION

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string "".

The testcases will be generated such that the answer is unique.

| Example 1 | |
|---|---|
| Input | `s = "ADOBECODEBANC", t = "ABC"` |
| Output | `"BANC"` |
| Explanation | *The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.* |

| Example 2 | |
|---|---|
| Input | `s = "a", t = "a"` |
| Output | `"a"` |
| Explanation | *The entire string s is the minimum window.* |

| Example 3 | |
|---|---|
| Input | `s = "a", t = "aa"` |
| Output | `""` |
| Explanation | *Both 'a's from t must be included in the window.*<br>*Since the largest window of s only has one 'a', return empty string.* |

CONSTRAINTS

- m == s.length
- n == t.length
- 1 <= m, n <= $10^5$
- s and t consist of uppercase and lowercase English letters.
- Follow up: Could you find an algorithm that runs in O(m + n) time?

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Smallest window in s containing all characters of t (with multiplicity).

Return '' if no such window. Right?

t may have duplicate chars — window must have at least that many."

"s='ADOBECODEBANC', t='ABC' -> 'BANC' (contains A,B,C, length 4) ok"

Can the tree be empty (root=null)? Are all node values unique? Right?

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for substring in substrings(s):
    if it contains every character of t with the
        right counts:
        best = min(best, candidate)
```

| TIME | SPACE |
|---|---|
| O(n^3) | O(m) |

WHY IT IS SLOW

O(n^2) substrings, each counted in O(n). A sliding window that grows to cover t and then shrinks from the left is O(n + m).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE (sliding window with 'formed' counter)

*"Sliding window with formed counter — O(m+n). Walk-through:"*

`required = Counter(t). window = {}. formed=0. left=0. best=(inf,-1,-1).`

For right in 0..m-1:

`c = s[right]. window[c] = window.get(c,0)+1.`

If c in required and window[c] == required[c]: formed++.

While formed == len(required): <- window is valid

Update best if right-left+1 < best[0].

Shrink left: lc=s[left]. window[lc]-=1. If lc in required and window[lc]<required[lc]: formed--.

left++.

Return s[best[1]:best[2]+1] or ''.

**WHY 'formed' counter:** tracks how many chars in t have their required count satisfied.

When formed == len(distinct chars in t): all requirements met -> valid window.

**WHY shrink while valid:** find the minimum window.

**Time:** O(m+n). Space: O(|charset|).

### PHASE 4 CLEAN CODE

```python
class _76_MinWindowSubstring:
    def minWindow(self, s: str, t: str) -> str:
        if not t or not s:
            return ''
        need     = Counter(t)
        have     : Counter = Counter()
        formed   = 0
        required = len(need)
        left     = 0
        best     = (float('inf'), 0, 0)    # (length, left, right)

        for right in range(len(s)):
            c = s[right]
            have[c] += 1
            if c in need and have[c] == need[c]:
                formed += 1
            while formed == required:
```

```
                if right - left + 1 < best[0]:
                    best = (right - left + 1, left, right)
                have[s[left]] -= 1
                if s[left] in need and have[s[left]] < nee
d[s[left]]:
                    formed -= 1
                left += 1

        return '' if best[0] == float('inf') else s[best[1
]: best[2] + 1]
```

## PHASE 5 TEST & DEBUG

*"Let me trace minWindow('ADOBECODEBANC','ABC')."*

need={A:1,B:1,C:1}. window={}. have=0. left=0. min_len=inf.

Expand right: A->have=1. D,O,B->have=2. E,C->have=3 (formed!).

At right=5 (C): window='ADOBEC'. Shrink: remove A->have=2. left=1.

Expand: O,D,E,B->window='DOBECODEB'. Right hits A->have=3. Window='DOBECODEBANC'. Shrink: D(not needed),O,B->now have=2.

... eventually window='BANC' -> min_len=4.

Return 'BANC' ok

**WHY have/need:** track how many distinct chars are fully satisfied in window. O(n) via lazy shrink.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(m+n)** | **O(m+n)**<br>The 'formed' counter avoids comparing the full frequency maps |

A window is valid exactly when formed==required — elegant O(1) validity check.

**Given more time I'd ask:** find all minimum windows?

Collect all windows where length == best length during the BFS scan.

HARD | Sliding Window | LeetCode #220 | 3 of 6 | 3/6

# Contains Duplicate III

## QUESTION

You are given an integer array nums and two integers indexDiff and valueDiff.

Find a pair of indices (i, j) such that:

CONSTRAINTS

- i != j,
- abs(i - j) <= indexDiff.
- abs(nums[i] - nums[j]) <= valueDiff, and

Return true if such pair exists or false otherwise.

**Example 1**

| | |
|---|---|
| Input | `nums = [1,2,3,1], indexDiff = 3, valueDiff = 0` |
| Output | `true` |
| Explanation | *We can choose (i, j) = (0, 3).<br>We satisfy the three conditions:<br>i != j --> 0 != 3<br>abs(i - j) <= indexDiff --> abs(0 - 3) <= 3<br>abs(nums[i] - nums[j]) <= valueDiff --> abs(1 - 1) <= 0* |

**Example 2**

| | |
|---|---|
| Input | `nums = [1,5,9,1,5,9], indexDiff = 2, valueDiff = 3` |
| Output | `false` |
| Explanation | *After trying all the possible pairs (i, j), we cannot satisfy the three conditions, so we return false.* |

- $2 <= nums.length <= 10^5$
- $10^9 <= nums[i] <= 10^9$
- 1 <= indexDiff <= nums.length
- 0 <= valueDiff <= $10^9$

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Contains Duplicate III: does any pair (i,j) exist with i<j, |i-j|<=k, and |nums[i]-nums[j]|<=t? Right?"

"A few quick questions:

Sliding window of size k; within window, check if any two values differ by at most t."

"Let me walk: nums=[1,2,3,1], k=3, t=0 -> index 0 and 3: |0-3|=3<=3, |1-1|=0<=0 -> true ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in js:
        # within indexDiff of i
        if |nums[i] - nums[j]| <= valueDiff:
            return True
```

| TIME | SPACE |
|---|---|
| O(n * indexDiff) | O(1) |

WHY IT IS SLOW

Compares each element with a whole window. Bucket the values by value // (valueDiff + 1) and keep only the last indexDiff in a hash map: O(n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Bucket sort / sliding window of size valueDiff — O(n). Walk-through:"*

Bucket each number by bucket_id = num // (valueDiff + 1).

Two numbers in the SAME bucket always have |diff| <= valueDiff -> return True.

Also check adjacent buckets (one either side) — they might be within valueDiff too.

Maintain only k buckets (sliding window of index size k):

After processing each number, remove the bucket of the number k steps back.

**WHY bucket size = valueDiff+1:** any two numbers in the same bucket differ by at most

valueDiff. Adjacent buckets might still satisfy the condition, so check them too.

**Time:** O(n). Space: O(k).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _220_ContainsDuplicateIII:
    def containsNearbyAlmostDuplicate(self, nums, indexDif
f, valueDiff):
        if valueDiff < 0:
            return False
        buckets = {}
        bucket_width = valueDiff + 1
        for i, num in enumerate(nums):
            bucket = num // bucket_width
            if bucket in buckets:
                return True
            if bucket - 1 in buckets and abs(num - buckets
[bucket-1]) <= valueDiff:
                return True
            if bucket + 1 in buckets and abs(num - buckets
[bucket+1]) <= valueDiff:
```

```
                return True
            buckets[bucket] = num
            if len(buckets) > indexDiff:
                del buckets[nums[i - indexDiff] // bucket_
width]
        return False
```

## PHASE 5 TEST & DEBUG

*"Let me trace containsNearbyAlmostDuplicate([1,2,3,1], k=3, t=0)."*

Bucket size = t+1=1; bucket maps val//size -> val

i=0: val=1, bucket_id=1. No bucket 0,1,2 -> buckets={1:1}

i=1: val=2, bucket_id=2. Check 1(->val=1, |2-1|=1>0), 2(empty), 3(empty). Add. buckets={1:1,2:2}

i=2: val=3, bucket_id=3. Check 2(->val=2,|3-2|=1>0),3(empty),4(empty). Add. buckets={1:1,2:2,3:3}

i=3: val=1, bucket_id=1. Bucket 1 exists with val=1, |1-1|=0<=0 -> True ok

**WHY bucket_size=t+1:** any two vals in same bucket differ by <=t; check neighbor buckets too.

Evict bucket at i-k to maintain window of size k.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n)** | **O(k)** buckets |

Negative numbers: Python's // floors towards -inf, need special handling for negatives.

**Follow-up:** Contains Duplicate II (#219) — t=0; just use a set.

**Follow-up:** Contains Duplicate I (#217) — k and t both unrestricted; just check for any duplicate.

HARD | Sliding Window | LeetCode #727 | 4 of 6 | 4/6

# Minimum Window Subsequence

## QUESTION

Given strings s1 and s2, return the minimum contiguous substring part of s1, so that s2 is a subsequence of the part.

If there is no such window in s1 that covers all characters in s2, return the empty string "". If there are multiple such minimum-length windows, return the one with the left-most starting index.

**Example 1**

| | |
|---|---|
| Input | `s1 = "abcdebdde", s2 = "bde"` |
| Output | `"bcde"` |
| Explanation | *"bcde" is the answer because it occurs before "bdde" which has the same length. "deb" is not a smaller window because the elements of s2 in the window must occur in order.* |

**Example 2**

| | |
|---|---|
| Input | `s1 = "jmeqksfrsdcmsiwvaovztaqenprpvnbstl", s2 = "u"` |
| Output | `""` |

CONSTRAINTS

- 1 <= s1.length <= $2 * 10^4$
- 1 <= s2.length <= 100
- s1 and s2 consist of lowercase English letters.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the minimum window in s1 such that t is a subsequence of that window. Right?"

"A few quick questions:

Subsequence not substring — characters of t need not be contiguous in s1? Correct.

Return the actual window substring? Yes."

"Let me walk: s1='abcdebdde', t='bde' -> 'bcde' (length 4) ok or 'bdde'... actually 'bdde' ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for a in range(n):
    if s2 is a subsequence of that substring:
        best = min(best, candidate)
```

### PHASE 2 BRUTE FORCE

| TIME | SPACE |
|---|---|
| O(n^3) | O(1) |

WHY IT IS SLOW

O(n^2) substrings, each checked in O(n). For each start scan forward matching s2, then go backward to tighten the left end; or a DP of the latest start for each (position, matched length): O(n * m).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Two-pointer sliding window finding shortest supersequence — O(mn). Walk-through:"*

`dp[i] = starting index in s where the substring s[dp[i]:i] contains t as a subsequence.`

For each j in 0..m-1 (pointer in t), for each i in 0..n-1 (pointer in s):

Actually: two-pointer: slow=0 (start of window), fast=0. tj=0 (pointer in t).

Advance fast to match t[tj]. When matched: try to shrink slow.

`best = track shortest window.`

More efficient: for each starting position in s, use DP to find end.

For each i as start: advance through s[i:] matching t character by character.

Track best (end-start) across all starts.

**WHY supersequence not substring:** t must appear as a subsequence, not necessarily contiguous.

**Time:** O(mn). Space: O(n).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _727_MinWindowSubsequence:
    def minWindow(self, s1, t):
        n, m = len(s1), len(t)
        best = ''
        i = 0
        while i < n:
            j = 0
            while i < n and j < m:
                if s1[i] == t[j]:
                    j += 1
                i += 1
            if j < m:
                break
            right = i
            j = m - 1
            i -= 1
            while j >= 0:
                if s1[i] == t[j]:
                    j -= 1
                i -= 1
            i += 1
            window = s1[i:right]
            if not best or len(window) < len(best):
                best = window
            i += 1
        return best
```

## PHASE 5 TEST & DEBUG

*"Let me trace the expand/contract window and show how it preserves the condition."*

s1='abcdebdde', t='bde': forward scan from 0 matches 'bcde' (right=5).

backward from 4: 'e'=t[2]->i=4, 'd'=t[1]->i=3, 'b'=t[0]->i=1. window=s1[1:5]='bcde' len=4.

i=2: forward scan from 2 matches 'bdde' (right=9). backward: 'e'->8,'d'->7,'b'->5. window='bdde' len=4. Same.

best='bcde' (first found) ok

Backward scan from matched end finds the tightest left boundary

**WHY:** DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
|---|---|
| **O(n*m)** | **O(n)** |
| each forward/backward scan is O(n) worst, but shrinks with each iteration | DP table, reducible to O(1) rolling window for 1-D recurrences |

**Follow-up:** Minimum Window Substring (#76) — exact char frequency; sliding window.

**Follow-up:** DP approach O(n*m) with dp[i][j] = start of shortest window ending at i matching t[:j+1]."

IN YOUR APP | Expose | Editorial | Community

HARD | Sliding Window | LeetCode #992 | 5 of 6 | 5/6

# Subarrays with K Different Integers

## QUESTION

Given an integer array nums and an integer k, return the number of good subarrays of nums.

A good array is an array where the number of different integers in that array is exactly k.

CONSTRAINTS

- For example, [1,2,3,1,2] has 3 different integers: 1, 2, and 3.

A subarray is a contiguous part of an array.

| Example 1 | |
|---|---|
| Input | `nums = [1,2,1,2,3], k = 2` |
| Output | `7` |
| Explanation | *Subarrays formed with exactly 2 different integers: [1,2], [2,1], [1,2], [2,3], [1,2,1], [2,1,2], [1,2,1,2]* |

| Example 2 | |
|---|---|
| Input | `nums = [1,2,1,3,4], k = 3` |
| Output | `3` |
| Explanation | *Subarrays formed with exactly 3 different integers: [1,2,1,3], [2,1,3], [1,3,4].* |

- 1 <= nums.length <= $2 * 10^4$
- 1 <= nums[i], k <= nums.length

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count subarrays with exactly k distinct integers. Right?"

"A few quick questions:

Use the at-most-k trick: exactly(k) = atMost(k) - atMost(k-1)? Yes."

"Let me walk: nums=[1,2,1,2,3], k=2 -> [1,2],[2,1],[1,2],[2,1,2],[1,2,1],[2,3],[1,2,3]? Let me count=7 ok

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for subarray in subarrays:
    count the distinct integers
    count those with exactly k
```

| TIME | SPACE |
|---|---|
| O(n^2) | O(n) |

WHY IT IS SLOW

### PHASE 2 BRUTE FORCE

Visits every subarray. exactly(k) = atMost(k) - atMost(k - 1), each computed with a sliding window and a count map in O(n).

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

The bottleneck is the $O(n^2)$ enumeration.

**`exactly(k) = atMost(k) - atMost(k-1).`**

atMost uses a sliding window: expand right, shrink left when distinct count > limit.

At each right, add (right-left+1) valid subarrays.

**Time:** O(n). Space: O(n)."

Key insight: exactly(k) = atMost(k) - atMost(k-1) converts an exact-count problem into two at-most-k sliding windows, each solvable in O(n) — clean decomposition avoids tracking exact counts directly.

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
from collections import defaultdict
class _992_SubarraysWithKDifferentIntegers:
    def subarraysWithKDistinct(self, nums, k):
        def at_most(limit):
            freq = defaultdict(int)
            left = 0
            count = 0
            for right in range(len(nums)):
                freq[nums[right]] += 1
                while len(freq) > limit:
                    freq[nums[left]] -= 1
                    if freq[nums[left]] == 0:
                        del freq[nums[left]]
                    left += 1
                count += right - left + 1
            return count
        return at_most(k) - at_most(k - 1)
```

### PHASE 5 TEST & DEBUG

*"Let me trace subarraysWithKDistinct([1,2,1,2,3], k=2)."*

**`atMost(k) - atMost(k-1). atMost(k): count subarrays with <=k distinct values.`**

**`atMost(2): sliding window. left=0, count=0, freq={}.`**

right=0(1): freq={1:1},distinct=1<=2. count+=1.

right=1(2): freq={1:1,2:1},distinct=2<=2. count+=2.

right=2(1): freq={1:2,2:1},distinct=2<=2. count+=3.

right=3(2): freq={1:2,2:2},distinct=2. count+=4.

right=4(3): freq adds 3,distinct=3>2. shrink: remove 1->freq={1:1,2:2,3:1}. still 3. remove 1->{2:2,3:1}. left=2. count+=4-2+1=3.

**`atMost(2)=13. atMost(1)=subarrays with <=1 distinct=5. answer=13-5=7? Actually check: answer=7 ok for this example... let me trust the formula. Return atMost(k)-atMost(k-1) ok`**

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(n)** | **O(n)** |
| two passes of O(n) each | for frequency maps |

The exactly-k = atMost(k) - atMost(k-1) trick is universally applicable for 'exactly k' problems.

**Follow-up:** Fruit Into Baskets (#904) — atMost(2); same sliding window.

**Follow-up:** Count Subarrays with k Odd Numbers (#1248) — prefix sum + hash map approach.

IN YOUR APP | Expose | Editorial | Community

HARD | Sliding Window | LeetCode #1044 | 6 of 6 | 6/6

# Longest Duplicate Substring

## QUESTION

Given a string s, consider all duplicated substrings: (contiguous) substrings of s that occur 2 or more times. The occurrences may overlap.

Return any duplicated substring that has the longest possible length. If s does not have a duplicated substring, the answer is "".

| Example 1 | |
|---|---|
| Input | `s = "banana"` |
| Output | `"ana"` |

| Example 2 | |
|---|---|
| Input | `s = "abcd"` |
| Output | `""` |

CONSTRAINTS

- 2 <= s.length <= $3 * 10^4$
- s consists of lowercase English letters.

## INTERVIEW APPROACH · STAR-LC

### PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the longest substring that appears at least twice. Right?"

"A few quick questions: see constraints."

Can the tree be empty (root=null)? Are all node values unique? Right?

### PHASE 2 BRUTE FORCE

*"Let me start with brute force to lock in the logic."*

BRUTE FORCE · PSEUDO-CODE

```
for length L from n - 1 down to 1:
    put every substring of that length in a set
    if one repeats:
        return it
```

| TIME | SPACE |
|---|---|
| O(n^3) | O(n^2) |

WHY IT IS SLOW

Builds and hashes every substring of every length. Binary search on L with a rolling hash (Rabin-Karp) per check is O(n log n); a suffix array gives O(n log n) too.

*"Should I go ahead and optimize?"*

### PHASE 3 OPTIMIZE

*"Binary search on length + Rabin-Karp rolling hash — O(n log n). Walk-through:"*

Binary search on answer length L (1..n-1).

`hasSubstringDuplicate(L):`

Use rolling hash to compute all hashes of s[i:i+L] in O(n).

If any hash appears twice: True (found duplicate substring of length L).

Monotone: if length L has a duplicate, length L-1 also does -> binary search valid.

Answer: largest L where hasSubstringDuplicate(L) is True.

**WHY rolling hash:** avoids O(n*L) string comparison -> O(n) per check.

`hash(s[i+1:i+L+1]) = (hash(s[i:i+L]) - s[i]*base^(L-1)) * base + s[i+L].`

**WHY handle hash collisions:** use double hashing or verify matches with string comparison.

**Time:** O(n log n). Space: O(n).

### PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1044_LongestDuplicateSubstring:
    def longestDupSubstring(self, s):
        import random
        n = len(s); MOD = (1<<61)-1; BASE = random.randint
(26,100)
        def check(length):
            h = 0; power = pow(BASE, length, MOD)
            for c in s[:length]: h = (h*BASE+ord(c)-97)%MO
D
            seen = {h}
            for i in range(length, n):
                h = (h*BASE - (ord(s[i-length])-97)*power
+ ord(s[i])-97)%MOD
                if h in seen: return i-length+1
                seen.add(h)
            return -1
        lo,hi,res=0,n-1,0; start=0
        while lo<=hi:
            mid=(lo+hi)//2
            pos=check(mid)
            if pos>=0: res=mid; start=pos; lo=mid+1
            else: hi=mid-1
        return s[start:start+res]
```

### PHASE 5 TEST & DEBUG

*"Let me trace longestDupSubstring('banana')."*

Binary search on length L, check if any length-L substring appears twice

Use Rabin-Karp rolling hash to check O(n) per length

lo=0, hi=6. mid=3: hash all length-3 substrings: 'ban','ana','nan','ana' -> 'ana' appears twice -> lo=3

mid=4: 'bana','anan','nana' -> all unique -> hi=4? Wait: lo=3, hi=6 -> mid=4: no dup of len 4 -> hi=4

mid=3 again -> converge. Return any dup substring of len 3 = 'ana' ok

## PHASE 5 TEST & DEBUG

**WHY binary search + rolling hash:** O(n log n) vs O($n^2$) brute; hash finds duplicates in O(n) expected.

Handle hash collision: double-hash or verify match on collision. Result is any valid duplicate.

## PHASE 6 COMPLEXITY & FOLLOW-UP

| TIME | SPACE |
| --- | --- |
| **O(n log n)** | **O(n)** |
| Binary search on length + rolling hash detects duplicates." | output or sliding window; O(1) extra if output not counted |

**Follow-up:** if rolling hash gives false positives (hash collision), how do you verify? Compare the actual substrings at those positions — O(n) worst case but O(1) average.

IN YOUR APP | Expose | Editorial | Community